Why OpenAI's query separation is nearly quartic

An examples-first explanation of the optimal randomized–quantum exponent

Will Cook · November 2026

1. Two searches, one inside the other

Does an $m \times m$ bit matrix have an all-one column? Hide a zero in a random row of each column, independently, then erase the zero in a uniformly chosen column. The answer changes. Fix h rows and inspect them in every column on both matrices. The mh queries see the difference precisely when the erased zero was in one of those rows, with probability h/m. Adaptive choices do not improve the order of this bound: Section 5 shows that G queries find the erased bit with probability at most $4G/m^2$. Constant advantage costs order m^2 classical queries.

A quantum algorithm can search m candidates in about $\sqrt{m}$ tests. Testing whether one column is all ones means searching *its* m rows for a zero. Thus this use of quantum search puts one square root inside another and costs about m bit queries, up to logarithmic factors (Aaronson et al. 2016, sec. 3.1). Suppose instead that every column could point to a zero in every rival. Two of them could then contest just their claims against one another. An honest all-one column would exhibit a zero in its opponent, while every pointer back into it must find a one.

OpenAI uses these contests to find a short list of candidates before checking anyone's full certificate. The two searches can then be paid for separately. The certificate has to be useful to the quantum algorithm without giving away the winning column classically. Its location in each column is therefore determined by several answers that are costly to compute classically. The new function asks whether some column has an all-one string **and** a valid certificate at its true address. It is total: every assignment of the input bits has an answer.

Write R and Q for the randomized and quantum query complexities. A randomized query reads a specified bit; a quantum query can address positions in superposition. Computation between calls is free. Both complexities count the least worst-case budget giving error at most $1/3$ on each input. For a partial function, only inputs in its *promise* have prescribed answers; the query cap still applies elsewhere.

For every fixed integer $k \geq 2$ and all sufficiently large powers of four m, OpenAI's total function satisfies (OpenAI 2026, Theorem 1.1)[1]

$$Q(F_{k,m}) = \widetilde{O}_k(\sqrt{m}), \qquad R(F_{k,m}) = \widetilde{\Omega}_k(m^{2-1/k}).$$

Here m counts columns and k counts blocks in each address input. The tildes suppress powers of $\log m$. Ignoring those logarithms, $m^{2-1/k} = (\sqrt{m})^{4-2/k}$. Choosing a large fixed k rules out any proposed universal exponent below four. Four suffices by the theorem of Aaronson, Ben-David, Kothari, Rao and Tal (Aaronson et al. 2021).

We use finite-dimensional linear algebra, conditional probability and asymptotic notation. The square-root saving in search follows from the two reflections in Section 3.2. General accounts of the query model include (Buhrman and de Wolf 2002, sec. 3; de Wolf 2026, chaps. 1, 2 and 7).

[1]The theorem and construction are OpenAI's. This exposition and its Lean development were prepared with AI systems under the author's direction.

2. A contest without an ordering

Start with the top row of Figure 1. Its four strings are 1111, 0111, 1011 and 1101. The three solid arrows from the shaded cell point to $X_2[1] = 0$, $X_3[2] = 0$ and $X_4[3] = 0$. Now follow the dashed arrow back from column 3. Wherever it landed in column 1 it would find a one. Column 1 can disqualify every rival, and no rival can disqualify it.

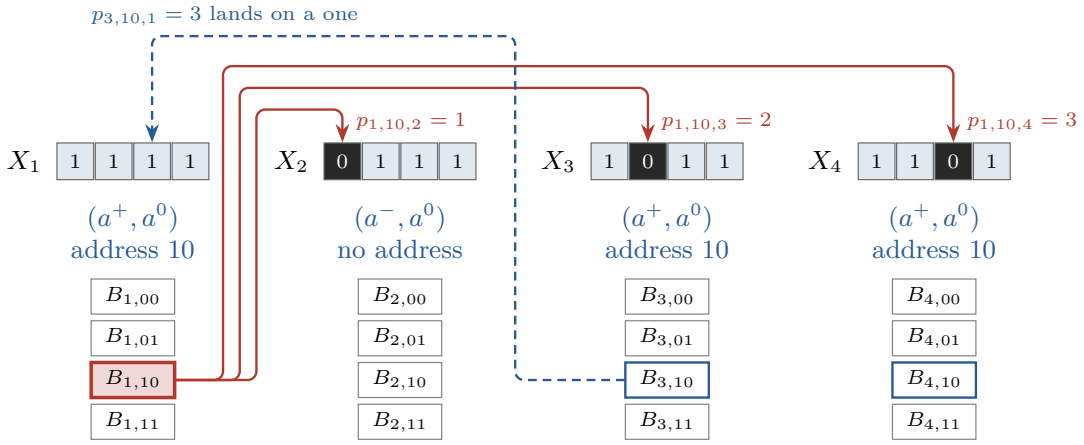


Figure 1: Trace the three solid pointers from column 1's cell 10 to zeros in the other columns. The dashed pointer from column 3 to column 1 finds a one.

A pointer names a row in a *different* column. An address names a cell in the column's *own* array. Here each column has four cells, labelled $00, 01, 10, 11$. The pair beneath each string gives two inputs to a partial function. In column 1 the pair (a^+, a^0) has answers $(1, 0)$, selecting cell 10. In column 2 the first input a^- has no answer, so no cell is selected. At full size the address has $20 \log_2 m$ bits rather than the two drawn here.

To win, a column must be all ones. Its true cell must justify its address and point to a zero in every other column. Section 4 gives the local tests that make "justify" precise. All cell contents are input bits, so even an unselected cell can contain convincing claims. At most one column can win: if two did, each would need a pointer to a zero in the other's all-one string.

Two columns meet. Let column 1 propose its true cell 10 and let its rival propose any label. Read each proposed cell's pointer into the other column. A zero disqualifies the column where it lies. If both or neither are disqualified, take the smaller index. Against the true cell of column 1, column 2 loses under *any* label it proposes. Its own pointer has no zero to find.

A false pointer. Keep the four strings in the figure, but let the pointer from cell 10 of column 1 land on the *second* bit of column 2, a one. Column 1 still beats 2, now by the index rule, and still beats 3 and 4. But it has lost its valid certificate. The answer on this board is zero, however impressive the comparison record of column 1.

A cycle. On the board of Figure 2 the four strings are as in Figure 1, and column 1 wins at 10 with its three honest pointers. Among the other columns, 2's pointer into 3 finds the zero at row 2, while 3's pointer back into 2 finds a one. Hence 2 beats 3. The same contest gives 3 over 4 and 4 over 2. These pointers are the same in every cell of their column, so even a meaningless label gives the indicated result.

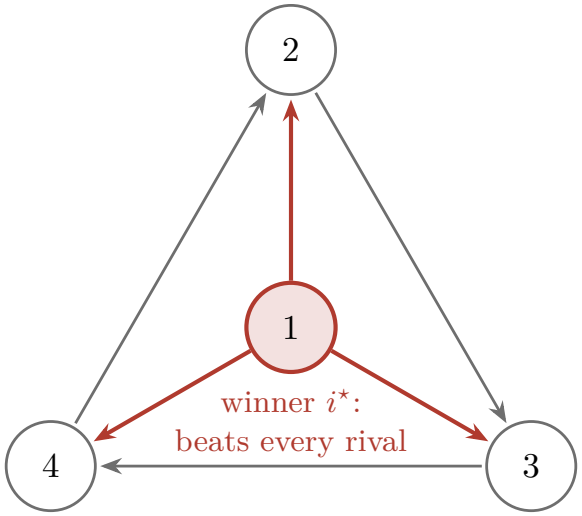


Figure 2: An arrow goes from the victor to the loser. The three listed pointers reach zeros. In the reverse direction, columns 3, 4 and 2 point respectively to rows 2, 1 and 1 of columns 2, 3 and 4, all ones.

Choose one column uniformly as a *pivot*, keep it on a list, and consider next only those that beat it. Try pivot 2 in Figure 2. Columns 1 and 4 survive. The four choices $1, 2, 3, 4$ leave $0, 2, 2, 2$ survivors, an average of $3/2$. The case with no survivors is a success: we have already listed column 1.

Would reversing one of the cycle's arrows change the average? Across all pivot choices every unordered pair contributes just one survivor: whichever member beats the other survives when the other is pivot. Reversing an arrow transfers that contribution to a different pivot. For *any* fixed tournament of m columns, the mean after a uniform first pivot is therefore $\binom{m}{2}/m = (m-1)/2$. We can make progress without sorting. What if two losing columns can give a different result every time they meet?

3. Finding a winner by comparisons

Suppose that, on the fixed input board, a fresh comparison of i and j declares exactly one of them the victor. Write a_{ij} for the probability it declares i. We require

$$a_{ii} = 0, \qquad a_{ij} + a_{ji} = 1 \quad (i \neq j), \qquad a_{i^\star j} \geq 1 - \varepsilon \quad (j \neq i^\star)$$

if a genuine winner $i^\star$ exists. Self-comparison loses by convention. Fresh calls use independent workspace. For the quantum implementation we will need to run the comparison on a superposition of column labels and to undo it. Section 4 will supply both operations at polylogarithmic query cost for fixed k.

On a board with a winner the search has one obligation: include it on the list with high probability. On a board without one, *any* list will do, as long as it is short. A separate verifier can reject the false certificates. The proof will therefore need no reliable answer between two losers.

3.1 Counting survival when the matches are random

In Figure 2 a survivor beat every earlier pivot. With random comparisons we can still ask how often column i would do so. Its weight w_i is the probability of beating all the pivots in fresh, independent comparisons. This measures survival in matches: the nonwinner with the false pointer could keep weight one. Initially every weight is one. To extend uniform choice among survivors, draw p with probability $w_p/|w|$, where $|w| = \sum_i w_i$. Keep p on the list. Beating it is one additional requirement, so $w'_i = w_i a_{ip}$; in particular the pivot's own weight vanishes. If every weight vanishes, keep the current list. This ideal experiment uses probabilities inaccessible to the algorithm.

Why should the ideal experiment make progress? For each unordered pair i, p there are two contributions to the expected remaining weight. Drawing p as pivot leaves $w_i a_{ip}$ from i; drawing i leaves $w_p a_{pi}$ from p. After multiplying by their respective selection probabilities, their combined contribution is $w_i w_p/|w|$, whatever happens in the comparison. For $|w| > 0$ this gives

$$\mathbb{E}[|w'| \mid w] = \frac{\sum_{i<p} w_i w_p}{|w|} = \frac{|w|^2 - \sum_i w_i^2}{2|w|} \le \frac{|w|}{2}.$$

The missing diagonal explains the -1 in $(m-1)/2$: a pivot cannot survive itself. For four equal initial weights the formula gives $(16-4)/8 = 3/2$, just as on our cycle. The same equality holds if all six comparisons between distinct pairs are fair coin flips. It is *total weight*, rather than the number of surviving columns, that the proof can halve.

Taking expectations halves the bound at each stage. Starting with total weight m gives $\mathbb{E}|w_T| \le m2^{-T}$ after T stages. Against any list of T or fewer other pivots the genuine winner still has weight at least $(1-\varepsilon)^T \ge 1 - T\varepsilon$. Take $T = \lceil \log_2(1000m) \rceil$ and $\varepsilon = 1/(1000T)$. An unlisted winner would account, by itself, for more than $1/2$ of a unit of weight. Hence

$$\Pr(\text{winner missing from the list}) \le \Pr(|w_T| > 1/2) \le 2m2^{-T} \le 1/500.$$

The same half-unit lower bound controls the cost of taking the *next* step. The pivot method for fixed orientations (Ambainis et al. 2026, sec. 4 and 6.1) would choose uniformly among the survivors. Here we have specified an ideal law $w_i/|w|$ in terms of unknown probabilities. How could an algorithm draw from it?

3.2 Sampling the next pivot

Choose i uniformly, compare it afresh with every pivot on the list, and accept it if it wins every comparison. For pivots $p_1, \ldots, p_j$,

$$\Pr(\text{accept and draw } i) = \frac{1}{m} \prod_{b=1}^{j} a_{ip_b} = \frac{w_i}{m}.$$

Summing over i gives the acceptance probability $q = |w|/m$. Thus $q > 1/(2m)$ as long as the winner is missing. Dividing each mass by q gives the desired pivot law $w_i/|w|$. The comparisons have done the weighting for us.

Try this with four columns and pivot 2, now letting the four win probabilities be $(1, 0, 1/2, 1/4)$ instead of the fixed arrows in Figure 2. Among successful trials, how often should column 3 appear? It is proposed a quarter of the time and accepted on half of those trials. The four masses of acceptance are $(1/4, 0, 1/8, 1/16)$, summing to $7/16$. Thus column 3 appears on $(1/8)/(7/16) = 2/7$ of successes. The full conditional law is $(4/7, 0, 2/7, 1/7)$. Uniform choice among the three positive labels would already differ on the second pivot.

Repeating the trial independently would take $1/q$ attempts on average. With one unit of weight left this is m attempts. The quantum sampler reduces the cost to order $\sqrt{m}$ while preserving the relative frequencies of the accepted labels.

A quantum state is a unit vector. Its squared coordinate at a basis label is the probability of obtaining that label on measurement (use the squared modulus for complex coordinates). Start with the uniform state $m^{-1/2}\sum_i |i\rangle$, where $|i\rangle$ denotes the basis vector labelled i. Run each column's comparisons against the pivot list in superposition, keeping the answers in separate registers without measuring them. Set a flag when all were won. For a fixed column the win probabilities multiply, so the squared mass on label i with flag one is exactly w_i/m.

Call the whole preparation A. For $0 < q < 1$, take as axes the unit directions g and b of its flag-one and flag-zero components, retaining the labels and workspace in both. Its prepared vector is $v = \sqrt{q}\, g + \sqrt{1-q}\, b$. Two reflections are available. Changing the sign whenever the flag is one reverses the g coordinate. Reflecting about v sends x to $2\langle v, x\rangle v - x$ in this real plane. To do the latter, undo A, reflect about the known starting vector, and apply A again (Brassard et al. 2002, Lemma 1; de Wolf 2026, chap. 7).

Try both reflections on Figure 2 after pivots 2 and 4. Only column 1 beats both, so $q = 1/4$. In (g, b) coordinates we start at $v = (1/2, \sqrt{3}/2)$. The flag reflection gives $x = (-1/2, \sqrt{3}/2)$. Now $\langle v, x\rangle = -1/4 + 3/4 = 1/2$, so reflecting about v gives $v - x = (1, 0)$. One pair of reflections has made acceptance certain.

For general q, write $v = \sin\theta\, g + \cos\theta\, b$. The first reflection sends angle θ from the b axis to $-\theta$; the second reflects it about the line at angle θ, taking it to 3θ. Each pair rotates through 2θ. After u pairs the state is

$$\sin((2u+1)\theta)\, g + \cos((2u+1)\theta)\, b.$$

The two coefficients change, but the relative label probabilities *within* g stay fixed. On acceptance we still obtain the law $w_i/|w|$, even when several labels have unequal weights. At $q = 1$ the initial state is entirely successful; at $q = 0$ there is no successful component.

When q is small, $\theta = \arcsin\sqrt{q}$ is of order $\sqrt{q}$. It takes order $1/\sqrt{q}$ pairs to reach an angle near $\pi/2$. Continuing past it can undo the gain. In the four-column calculation $\theta = \pi/6$ and a second pair gives success $\sin^2(5\pi/6) = 1/4$. Since q is unknown, we use a bounded schedule of random rotation counts. It uses $O(\sqrt{m}\log(1/\varepsilon))$ preparations and inverses on every input and fails with probability at most ε when $q \geq 1/(2m)$ (Boyer et al. 1998, Lemma 2). The bounded schedule is derived in Appendix A.3. Whichever count or fresh trial first succeeds, the conditional label law is the same.

Append each successful measured label, stopping on failure or after T pivots. Until the winner is listed, its weight exceeds $1/2$, so $q \geq 1/(2m)$ and the success guarantee applies. We can draw the same label in the ideal and actual processes whenever the actual sampler succeeds, because its conditional law is exactly right. Over at most T stages the chance of a failure before listing the winner is at most $T\varepsilon$. A failure after listing it leaves the winner safely on the list. There are at most T stages, and one preparation makes at most T comparisons. This gives $O(T^2\sqrt{m}\log(1/\varepsilon))$ comparison calls, counting inverses as well. How many bit queries does one such call require?

4. Computing an address and checking its certificate

Each comparison reads two outward pointers. A column first needs a cell label from which to read its pointer. For the four-column board this means producing the two bits that select a cell such as 10. An eight-bit example shows how the decoder can produce a label on every run, including runs on inputs outside the promise.

Forrelation measures agreement between one sign pattern and the Walsh transform of another (Aaronson and Ambainis 2018). In the four-row example, split eight bits into two blocks of four

and turn each bit x into the sign $(-1)^x$. With z_1, z_2 the resulting sign vectors, set

$$W = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}, \qquad H = W/2, \qquad \Phi = \langle z_1/2, H(z_2/2) \rangle .$$

The rows of W are orthogonal and have norm two, so H preserves lengths. For the common second block 1000, the sign vector is $g = (-1, 1, 1, 1)$ and $Wg = (2, -2, -2, -2)$. With $\delta = 2^{-10}$ at this size, the promised answer is 1 for $\Phi \geq \delta$ and 0 for $|\Phi| \leq \delta/2$; elsewhere it is undefined. We can read the three examples from the first block alone.

Address input	Relation of $z_1/2$ to $H(g/2)$	Φ	P
$a^+ = 0111\,1000$	equal	1	1
$a^0 = 0100\,1000$	orthogonal	0	0
$a^- = 1000\,1000$	opposite	-1	undefined

The winning column in Figure 1 uses (a^+, a^0), selecting cell 10. Column 2 uses (a^-, a^0) and has no true cell: a large negative correlation lies outside the promise.

A two-query experiment. One bit query flips an answer bit b at each position j where $x_j = 1$, extending the rule $|j, b\rangle \mapsto |j, b \oplus x_j\rangle$ linearly. Give it the answer state $(|0\rangle - |1\rangle)/\sqrt{2}$. A flip reverses the sign of this vector. Thus one query multiplies *each amplitude* at position j by $(-1)^{x_j}$, leaving the answer state available for reuse. Write D_z for this diagonal operation.

Start with $u = (1, 1, 1, 1)/2$. Query the second block, apply H, then query the first block. On a^+ the successive position states are

$$u \;\longmapsto\; \tfrac{1}{2}(-1, 1, 1, 1) \;\longmapsto\; \tfrac{1}{2}(1, -1, -1, -1) \;\longmapsto\; u.$$

Asking whether the final state lies along u succeeds with probability $|u^\mathsf{T} D_{z_1} H D_{z_2} u|^2 = \Phi^2$. Concretely, apply H once more and measure the first basis outcome, since $Hu = (1, 0, 0, 0)$. This last transform costs no queries. On a^0 the success probability is zero. On a^- the final state is $-u$ and the success probability is one. The decoder can therefore suggest the *same* first bit for columns 1 and 2 with certainty on our example. Only column 1 has a true address. Either proposal is fine during a comparison. Accepting a proposed winner will require evidence of a *defined* address.

The cost of a comparison. In the calculation above we queried a block, changed basis, then queried the next block. Repeating this alternation compares a chain of k sign patterns. In the full construction $m = 4^t$, $t \geq 1$, and each address input has k blocks of length m. The normalized Walsh matrix has entries $H_{ab} = (-1)^{a \cdot b}/\sqrt{m}$ for binary indices of length $2t$. With $u = (1, \ldots, 1)/\sqrt{m}$, put

$$\Phi = u^\mathsf{T} D_{z_1} H D_{z_2} H \cdots H D_{z_k} u, \qquad \delta = 2^{-5k}.$$

As before, the two promised ranges are $\Phi \geq \delta$ and $|\Phi| \leq \delta/2$. Now one trial uses k queries and projects with probability Φ^2. On promised inputs this probability is at least δ^2 or at most $\delta^2/4$. Independent trials therefore distinguish the two ranges with error η using $O_k(\log(1/\eta))$ queries on every input. The hidden constant depends on the gap and hence on k. Appendix A.5 gives the count.

For $\ell = \log_2 m$ the full board uses $r = 20\ell$ address inputs per column. Their answers select one of $2^r = m^{20}$ cells; a pointer within that cell uses ℓ bits to select a row of a rival. The board thus has polynomially many bits in m, although m counts only the columns. The addressed

array comes from cheat sheets (Aaronson et al. 2016, sec. 3.1), and direct pointers to zeros from pointer functions (Göös et al. 2018; Ambainis et al. 2017). OpenAI puts one such array and its outward pointers in every column (OpenAI 2026, sec. 2).

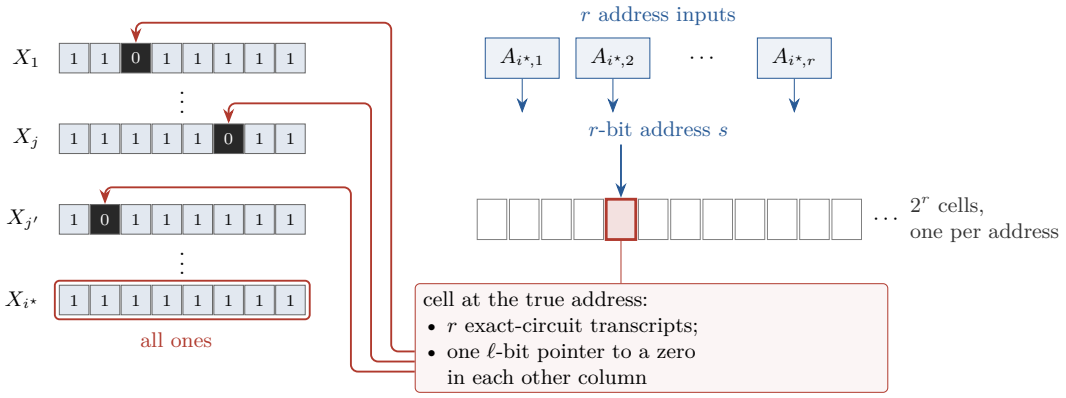


Figure 3: At full size each column has the same fields. On a winning column the true address answers select the cell shown. Its pointers reach into other columns' strings, while its circuit transcripts certify the address inputs in its own column.

Run the decoder on each address input with error ε/r on the promise. A winner therefore gets its whole address right except with probability ε. When it does, its true pointer finds a zero in its rival. Even if all the rival's address inputs are unpromised, its proposed cell cannot point to a zero in the winner. Ordering each pair by index before running the experiment gives complementary win probabilities however the pair is presented.

Keep each projection outcome in a fresh register, compute the proposed addresses and query the pointer bits and the two string bits they name. Finally record the contest result in a flag. Each step retains the information needed to run it backwards; reversing costs the same number of queries. The resulting controlled comparison therefore supplies Section 3's operation with $O_k(r\log(r/\varepsilon)+\ell)$ queries in either direction. It reads no circuit transcript.

The cost of verification. A public circuit can compute the signed numerator of Φ exactly by Walsh additions and subtractions, then compare it with the two thresholds. This takes $S = O_k(m\ell^2)$ gates of fan-in at most two, as shown in Appendix A.6. Its two output bits report membership in the promise and the value on the promise (zero outside it). A cell records its proposed values at every gate for each of its column's address inputs. The transcripts must be accurate, all domain bits one and the value bits equal to the cell label.

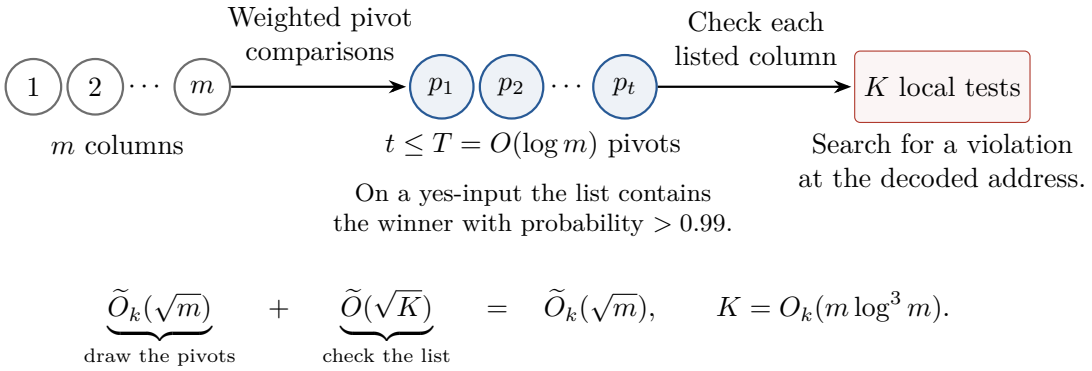
For a board where this distinction decides the answer, take column strings $0111, 1111, 1011, 1101$. Column 2's address inputs are (a^-, a^0). Every cell in column 2 stores their true circuit evaluations and points to the zeros in columns $1, 3, 4$. Try cell 00. This all-one column has honest pointers, its two value bits spell 00, and all its gate equations hold. Nevertheless the domain bits are $(0, 1)$. The first transcript certifies that a^- is *outside* the promise. The board has no winner.

Decode a listed candidate's address afresh. If a transcript in that cell contains a false gate value, the first such gate in computation order has correct predecessors and fails its local equation. Conversely, passing every gate test forces the entire evaluation to be correct. A nonwinner must therefore fail at least one test at *every* cell it might propose.

It suffices to test m string bits, $m-1$ pointers, rS gate equations and $2r$ output bits. Each gate test reads at most three bits. Following a pointer reads its ℓ bits and the named string bit. Thus there are $K = 2m-1+r(S+2) = O_k(m\ell^3)$ local tests, each costing $O(\ell)$ queries. All tests concern one proposed cell; its 2^r-1 neighbours cost nothing.

Search for a violation in that cell. A uniform test finds a violation with probability at least $1/K$ if any exists, so the same bounded amplification searches for one using $O(\sqrt{K}\log(1/\varepsilon))$ test evaluations. Accept the board if any listed column passes. On a 0-board every candidate, at any decoded address, has a defect. Each search misses a defect with probability at most ε. With at most T searches the total probability of a false acceptance is at most $T\varepsilon$. On a 1-board the list misses its winner with probability at most $1/500 + T\varepsilon$. Decoding the winner's address incorrectly adds at most ε more. At its correct address all tests pass.

The tournament uses $\widetilde{O}(\sqrt{m})$ cheap comparisons. Only its list of $T = O(\log m)$ candidates pays for a defect search, each at cost $\widetilde{O}_k(\sqrt{m})$. The two costs add (Figure 4). The full count gives $Q(F_{k,m}) \le C_k\sqrt{m}(\log_2 m)^6$.



$$\underbrace{\widetilde{O}_k(\sqrt{m})}_{\text{draw the pivots}} + \underbrace{\widetilde{O}(\sqrt{K})}_{\text{check the list}} = \widetilde{O}_k(\sqrt{m}), \qquad K = O_k(m\log^3 m).$$

Figure 4: A comparison follows one pointer in each direction. Only columns that reach the short list incur the cost of searching all their local tests for a defect.

5. A classical query to the hidden cell

What can a classical algorithm learn from the extra cells? Start with the losing board at the top of Figure 5. Each string has one zero, all address inputs are promised, and all cells are blank. Erase one string's zero and fill its true cell with accurate transcripts and pointers. We have made a winner. An algorithm that encounters neither change gives the same answer on both boards.

The erased bit presents the matrix-search problem from Section 1. The filled cell could give the game away: if its address in each column were public, reading the first domain bit in all m selected cells would distinguish the boards. Every blank cell gives zero, whereas the planted transcript gives one.

In the two-bit picture, stop the algorithm just as it asks for bit u of cell 10 in column i. **Before seeing the bit**, it has named two answers: if this is the true cell, those answers must be $(1, 0)$. A computation of the answers, a guess, or a choice based on all the preceding replies can lead to this query. In each case the coordinate itself serves as the prediction. On the full board it predicts all r answers.

How well can an adaptive classical algorithm predict those answers? For the same k-block promise problem whose quantum decoder we just used, Bansal and Sinha prove a randomized query lower bound (Bansal and Sinha 2021, Theorem 1.3 and Corollary 1.4). For each fixed k and all sufficiently large m it gives

$$R(P) > L, \qquad L = \left\lfloor m^{1-1/k}/(\log_2 m)^2 \right\rfloor \ge 1.$$

Finite minimax then gives a single distribution μ on promised address inputs under which every deterministic L-query procedure has success at most $2/3$ (Yao 1977). Draw every address input

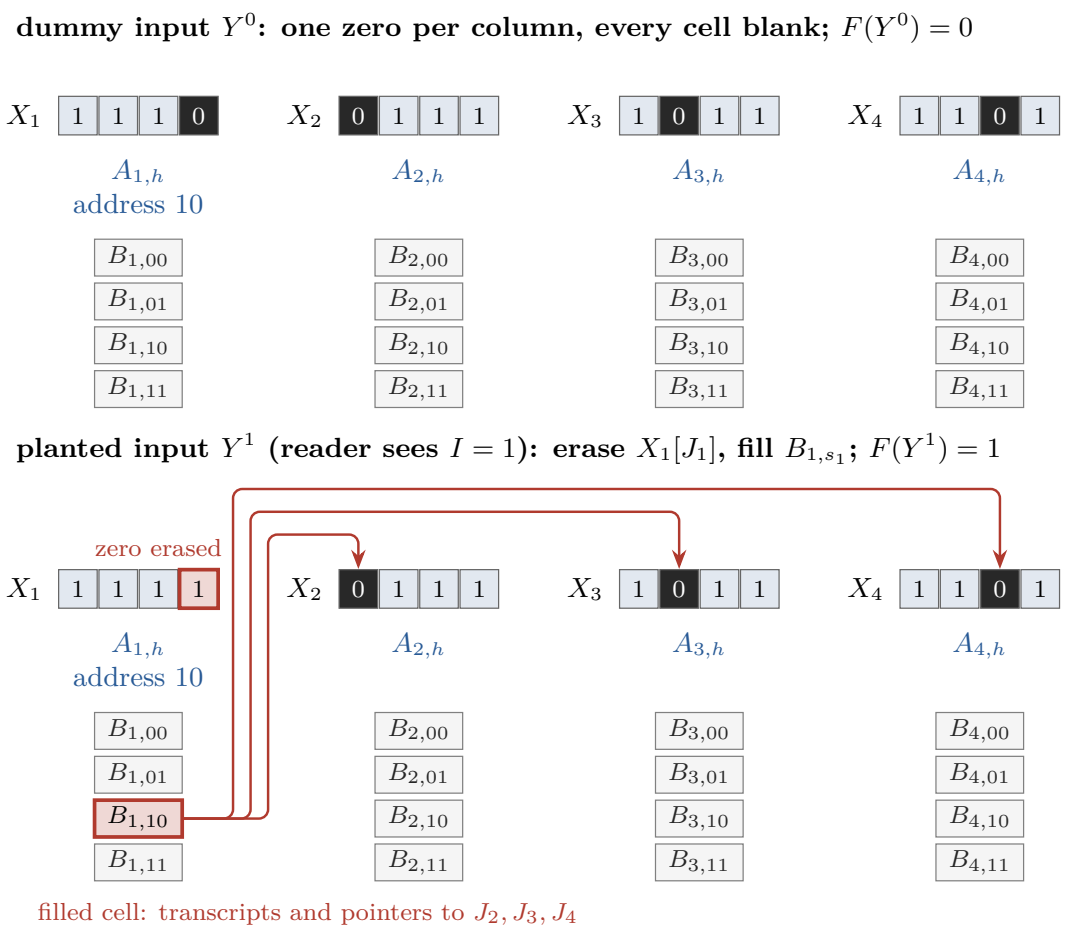


Figure 5: Planting column 1 in a four-column board. The address inputs stay the same. Erase the zero in row 4 of column 1 and fill cell 10 with the certificates and pointers it needs. The two runs see identical answers until they query a changed bit.

independently from μ and choose each zero location independently and uniformly. This gives the losing board. Choose the planted column I uniformly and independently of that entire board.

We count visits on the *losing* run. On the winning board a pointer from the planted cell might reveal another zero location. On the losing board all cells are blank and no such information is available. Once we fix the algorithm's coins, it makes identical choices on the two boards up to the first differing reply. It suffices to charge any visit on the losing run to the erased zero or the true cell, even if a queried cell bit happens to agree. Write G for the total query budget.

The zero. Until the zero of a column is discovered, it is uniform among that column's unread positions: changing it between any two of them preserves the whole transcript so far. While fewer than half of its positions have been read, a new string query therefore finds a zero with probability at most $2/m$. There are at most $2G/m$ such discoveries in expectation. Late discoveries need at least $m/2$ previous reads in their own column, affordable in at most $2G/m$ columns. Thus the expected number of discovered zeros is at most $4G/m$. Averaging over the independent I gives at most $4G/m^2$ for a hit on the selected zero.

The address. A column is heavy if the whole losing run spends more than L queries on its address inputs. At most G/L columns can be heavy; the chance that I is one of them is at most $G/(mL)$. For a light column, use the algorithm itself to predict the address as follows.

Fix a column i and a query time j. Given i's hidden tuple of r address inputs, sample the rest of the losing board independently: other columns' inputs, all zero locations and blank cells. Run

the algorithm. Answer every outside query from the simulated board and forward queries to the target tuple, stopping *before* its $(L+1)$st read. If the jth query names a bit of cell (i, s), return s without making the query. In every other case return a fixed address. Even after the last permitted target read you can still answer queries outside the tuple or intercept a cell coordinate. In the worked example in Appendix C, one branch stops before its second target read and the other reaches query four and outputs 10.

On every run where i is light this simulator can reproduce the choices up to time j. Hence

$$\{i \text{ light and query } j \text{ in its true cell}\} \subseteq \{\text{the capped predictor gets all } r \text{ answers right}\}.$$

Keep lightness *inside* the event on the left. Conditioning the input law on a light run might select especially easy inputs; the predictor on the right uses the original independent samples.

The separate-cap direct-product lemma bounds the right-hand probability by $(2/3)^r$ (Shaltiel 2003; Drucker 2012). In general, if success on one sample with L queries is at most β, a procedure with a cap of L on *each* of r independent samples and with independent side information gets all answers right with probability at most β^r. Our predictor has an even smaller cap of L reads across the entire tuple. Interleaving causes no loss in the bound: given a fixed transcript and the independent side information and coins, the remaining input laws factor. Each query changes the best remaining success probability for just one input, and cannot increase it on average, since making that query was already an available strategy. The product of the best remaining success probabilities consequently has nonincreasing expectation. Appendix B makes the argument precise.

Sum the light-visit bound over at most G query times for a fixed column, then *average* over the selected column I. The address length balances polynomially many cells against polynomially many guesses: $r = 20 \log_2 m$ gives $2^r = m^{20}$ but $(2/3)^r \le m^{-10}$. We can now account for every way of encountering a change.

Event on the losing run	Upper bound
The selected zero is discovered.	$4G/m^2$
The selected column is heavy.	$G/(mL)$
A query visits the true cell of the light selected column.	$G(2/3)^r$

At $G = \lfloor mL/100 \rfloor$ the sum is at most $4/100 + 1/100 + 1/(100m^8) < 1/3$, since $L \le m$. Averaging over the algorithm's coins preserves this bound. Yet a successful algorithm accepts with probability at least $2/3$ on the planted boards and at most $1/3$ on the losing boards; its outputs must differ in our paired experiment with probability at least $1/3$. Hence $R(F_{k,m}) > \lfloor mL/100 \rfloor$.

6. Why the exponent is four

The universal exponent asks for a *single* power α and constant C such that $R(f) \le C(1+Q(f))^{\alpha}$ for every total Boolean function f. Write α_{tot} for the infimum of such nonnegative exponents. To see how the construction rules a power out, try $\alpha = 3$. With $k = 3$ the classical exponent in m is $2 - 1/3 = 5/3$, whereas cubing the quantum bound gives only $3/2$. The surplus $1/6$ of a power of m eventually outweighs any fixed power of its logarithm.

For any $0 \le \alpha < 4$, choose $k \ge 2$ with $1/k < (4-\alpha)/2$, then hold k fixed. The cost of decoding and the size threshold for classical hardness both depend on that choice. For large m, taking the floor in L costs at most a factor of two. Using the same board and the same address circuit in the two estimates gives, with room in the numerical coefficient,

$$Q(F_{k,m}) \le C_k \sqrt{m}\,(\log_2 m)^6, \qquad R(F_{k,m}) \ge \frac{m^{2-1/k}}{400(\log_2 m)^2}.$$

The ratio of the two sides then satisfies

$$\frac{R(F_{k,m})}{(1+Q(F_{k,m}))^{\alpha}} \;\ge\; c_{k,\alpha}\,\frac{m^{2-1/k-\alpha/2}}{(\log_2 m)^{2+6\alpha}} \;\longrightarrow\; \infty.$$

The surplus power is positive by our choice of k. No uniform constant C can work. In the other direction, the universal bound of Aaronson, Ben-David, Kothari, Rao and Tal, building on Huang's sensitivity theorem (Aaronson et al. 2021; Huang 2019), gives deterministic query complexity $D(f) = O(Q(f)^4)$ for total functions. Since $R(f) \le D(f)$, four suffices. Thus $\alpha_{\mathrm{tot}} = 4$.

On the bare matrix the quantum algorithm tested each candidate inside a search over candidates. The outward pointers let it ask a cheaper question there: *which of these two columns can exhibit a zero in the other?* Finding the list and checking the list now cost about $\sqrt{m}$ apiece, and their costs add. The classical algorithm faces the other consequence of the same construction. To obtain even one bit from the certificate that changed, it must first name an address determined by independent hard answers. This preserves a cost of order mL, close to m^2 for large fixed k.

Appendix A. Supporting calculations

A.1 Counting the quantum queries

The quantum argument in Sections 3 and 4 uses the following count. It charges the same comparisons and local tests on every branch; early stopping can only reduce the cost.

A coherent test evaluation works on a superposition of test labels. It computes the next input position reversibly and retains earlier answers. To implement a read that is active only on some branches, query x_j into a temporary bit, conditionally copy it to the answer bit, then query j again to clear the temporary bit. This uses two ordinary oracle calls; on an inactive branch the answer and temporary bits are both restored. Padding to a common number of these controlled reads therefore has a fixed cap even in superposition. Running the computation backwards then restores its workspace. This implements the same tests using $O(\ell+1)$ bit queries per evaluation; the controlled-read construction gives the controlled implementation. No full transcript is read during a single test.

The count. We bound the longest run, including randomized iteration counts. A comparison uses $O_k(r\log(r/\varepsilon))$ queries for its decoders and $O(\ell+1)$ for its pointers, including the constant overhead of a reversible implementation. We count the tournament first, then verification.

Each stage uses $O(\sqrt{m}\log(1/\varepsilon))$ preparations, and each preparation makes at most T comparisons. There are at most T stages. Multiplying these three factors bounds the entire tournament by $O(\sqrt{m}\,T^2\log(1/\varepsilon))$ comparisons.

Verification makes at most T fresh address decodings. For each candidate it searches the K local tests, using $O(\sqrt{K}\log(1/\varepsilon))$ test evaluations. Thus the list requires $O(T\sqrt{K}\log(1/\varepsilon))$ test evaluations in all.

In bit-query units the two costs add:

$$O_k\Big(\sqrt{m}\,T^2\log(1/\varepsilon)[r\log(r/\varepsilon)+\ell]+T[r\log(r/\varepsilon)+\sqrt{K}\log(1/\varepsilon)\ell]\Big).$$

The first bracket charges comparisons; the second charges fresh address decoding and local-test search. Here $r, T, 1/\varepsilon = O(\ell)$ and $K = O_k(m\ell^3)$. Bounding $\log(1/\varepsilon) = O(\ell)$, $\log(r/\varepsilon) = O(\ell^2)$ and $\sqrt{K} = O_k(\sqrt{m}\,\ell^2)$ deliberately enlarges the logarithms and square root. These substitutions give $O_k(\sqrt{m}\,\ell^6)$ for the tournament and $O_k(\sqrt{m}\,\ell^5)$ for verification (the explicit count). Therefore

$$Q(F_{k,m}) \le C_k\sqrt{m}\,(\log_2 m)^6.$$

The tournament and short-list verification costs add, in place of the earlier nested search product $\sqrt{m}\sqrt{K}$. Here m counts columns. The exponent six is an artefact of this loose count. Square-root dependence comes from amplifying pivot sampling and searching the local tests. The short list limits how many verifications are needed; it is the quantum half of the exponent calculation.

A.2 Why the balanced AND–OR recipe stops at a cube

In the cheat-sheet construction (Aaronson et al. 2016, sec. 3.1), each of the n input bits of a partial function g is replaced by the answer to its own AND–OR instance on n^2 bits, for n^3 bits overall before adding certificates. In one instance, write the bits as n rows of n bits; its answer is 1 exactly when every row contains a 1. A 1-certificate selects one 1 in every row; a 0-certificate selects one entire zero row. Thus either answer has a certificate of n bits. Reading those certificates determines the n-bit vector supplied to g; the fixed domain predicate of g can then be checked by free computation. The certificate reads remain charged.

Up to logarithmic factors, the cited composition and cheat-sheet argument supplies randomized lower bound $R(g)n^2$ and quantum upper bound n when $Q(g)$ is polylogarithmic in n. Reading all n bits gives $R(g) \le n$, so this calculation reaches at most a cubic separation. Ben-David's earlier Forrelation construction (2015) and cheat sheets first gave power 5/2; Tal (2020) reached

$8/3-o(1)$, and the nearly linear partial-function bounds of Bansal and Sinha (2021) and Sherstov, Storozhenko and Wu (2023) gave $3-o(1)$. In the present construction, an address instance has about m bits while the quantum search costs about $\sqrt{m}$. The partial-function hardness can therefore contribute almost two powers of the quantum cost. Section 3 explains how comparisons make that square-root search possible.

The search problem in Section 3 also differs from quantum minimum-finding, which uses an order (Dürr and Høyer 1996), and from search with bounded-error tests for fixed Boolean answers (Høyer et al. 2003). The cycle and the fair comparison respectively rule out these two descriptions of the matches.

Certificate complexity is a separate measure: it counts the bits needed to force a function value. Ben-David and Kothari construct total functions for which randomized evaluation beats this measure (Ben-David and Kothari 2026, secs. 2–3). Their card encoding and the fixed-graph search cited in Section 3 are related constructions, not inputs to the randomized–quantum bound proved here.

A.3 The amplification rotation and a finite schedule

For $0 < q < 1$, write $A|0\rangle = \sin\theta\, g + \cos\theta\, b$ as in Section 3.2. Here A includes the index, all comparison registers and the final success flag. To reflect about $A|0\rangle$, apply A^{-1}, reflect about $|0\rangle$, and apply A again. Retaining the decoder and pointer-read registers makes this inverse possible and gives it the same query cost as A. The two reflections act on the plane spanned by these orthonormal success and failure vectors. In the ordered orthonormal basis (g, b), the success phase flip and the reflection about $A|0\rangle$ have matrices

$$S = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad R_{A0} = \begin{pmatrix} -\cos 2\theta & \sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}.$$

Their product $R_{A0}S = \left(\begin{smallmatrix} \cos 2\theta & \sin 2\theta \\ -\sin 2\theta & \cos 2\theta \end{smallmatrix}\right)$ sends $(\sin\theta, \cos\theta)$ to $(\sin 3\theta, \cos 3\theta)$. This calculation gives the rotation formula used in Section 3.2. It changes the total successful amplitude, leaving the normalized successful state unchanged.

Section 3 needs both a cap on every input and the conditional label law of Section 3.2. Set $M = \lceil 10/\sqrt{\gamma} \rceil$ and $N = \lceil \ln(1/\eta)/\ln(40/31) \rceil$. In each fresh trial use zero iterations with probability $1/2$, and otherwise choose u uniformly from $0, \dots, M-1$. Stop at the first acceptance, or after N failures. Each trial uses at most $2M+1$ calls to A or A^{-1}, so $N(2M+1)$ is an all-input cap.

To bound the failure probability, first suppose $\gamma \le q \le 1/2$. Averaging the rotation formula gives (Boyer et al. 1998, Lemma 2)

$$\frac{1}{M}\sum_{u=0}^{M-1} \sin^2((2u+1)\theta) = \frac{1}{2} - \frac{\sin(4M\theta)}{4M\sin(2\theta)} \ge \frac{1}{2} - \frac{1}{4M\sqrt{2\gamma}} \ge \frac{9}{20},$$

since $\sin(2\theta) = 2\sqrt{q(1-q)} \ge \sqrt{2\gamma}$. Thus the randomized-iteration branch alone contributes at least $9/40$ to the trial's acceptance probability. If $q \ge 1/2$, the zero-iteration branch contributes at least $q/2 \ge 1/4$. In either case, N fresh trials all fail with probability at most $(31/40)^N \le \eta$. Each successful trial has the label law derived in Section 3.2, so stopping at the first success preserves it. If $q < \gamma$, the cap still holds, although the acceptance guarantee need not.

A.4 The small circuit and its local tests

Optional circuit census. The explicit circuit for the four-column board has $S = 180$ non-input gates, so each candidate has $K = 4 + 3 + 2(180 + 2) = 371$ tests. In its zero-based numbering, input bits 6 and 7 feed gates 1 and 2: they compute $u = \neg x$, then $v = u \vee y$. On a^+ these last two input bits are both zero. To test gate 2, read the proposed u, the actual y and the proposed v. Reading $(1, 0, 0)$ exposes a false gate value, since $1 \vee 0 = 1$. The domain and value outputs are gates 178 and 179. Every cell contains 6 pointer bits and 360 transcript bits. With four cells in each of four columns, the full board has $4[4 + 16 + 4(6 + 360)] = 5936$ input bits. Only one cell enters the tests for a given candidate. This toy uses $r = 2$; the separation family uses $r = 40$ when $m = 4$, and its classical lower bound requires much larger m.

A.5 Repeating the projection experiment

For any requested error $0 < \eta < 1/2$, the projection experiment of Section 4 extends to every k. Phase kickback uses one query per sign diagonal, so a trial uses k queries and has projection probability $p = \Phi^2$. On the promise, either $p \geq c = \delta^2$ or $p \leq c/4$. Start each trial in fresh registers and repeat independently

$$\rho = \lceil 16c^{-1} \ln(1/\eta) \rceil$$

times; accept if at least $\rho c/2$ projections succeed. We use the following standard Chernoff estimates for the number Z of successes in these independent Bernoulli trials:

Promised case	Wrong decision	Tail bound used
$p \geq c$	$Z < \rho c/2$	At most $\exp(-\rho c/8)$, hence at most $\exp(-\rho c/16)$.
$p \leq c/4$	$Z \geq \rho c/2$	At most $\exp(-\rho c/16)$, by the upper tail at $p = c/4$ and monotonicity.

Thus each error is at most η. The cap is $k\rho = O_k(\log(1/\eta))$ on every input. The subscript permits constants depending on k, here including $c^{-1} = 2^{10k}$: cheap means fixed k and growing m. For a coherent comparison we retain the trial outcomes instead of measuring them; their measurement probabilities and independence remain the same.

A.6 Exact certification and the input size

The circuit uses integer arithmetic. The two factors $1/\sqrt{m}$ in u and the $k - 1$ internal Walsh factors give $\Phi = N/2^{t(k+1)}$ for an integer N. A fast Walsh transform recursively transforms two half-size vectors, then adds and subtracts their corresponding entries. These split-and-combine steps compute the numerator using additions and subtractions, and integer comparisons decide the two thresholds exactly. For example, $\Phi \geq 2^{-5k}$ is equivalent to $2^{5k}N \geq 2^{t(k+1)}$; the small-value test is $2^{5k+1}|N| \leq 2^{t(k+1)}$. Words of $O(k \log m)$ bits suffice, including the threshold shifts. The record's circuit construction gives the arithmetic and gate count. The quantum verifier checks a stored computation of this circuit; it does not run the entire circuit on each candidate.

The hardness allowance. This is the consequence of the randomized lower bound of Bansal and Sinha for this one-sided promise (2021, Theorem 1.3 and Corollary 1.4). For fixed k, the source bound is at least a positive constant depending on k times $m^{1-1/k}/(\log_2(km))^{1-1/k}$. Dividing this by the unfloored expression for $L_{k,m}$ gives a factor of order $(\log m)^{1+1/k}$, which tends to infinity and absorbs the constant.

Input length. A column contains m string bits, rkm address-input bits, and 2^r cells, each containing $(m - 1)\ell$ pointer bits and rS transcript bits. Thus the general function has $n = m[m + rkm + 2^r((m - 1)\ell + rS)]$ input bits, which is $O_k(m^{22}(\log m)^3)$ since $2^r = m^{20}$. Thus

the total input length is polynomial in m, although much larger than m. The three bracketed contributions are the column string, its address inputs, and its cell array. Here n is the actual input length; m controls the displayed query bounds. The comparison is between R and Q for this one function, with the same circuit; neither bound counts the time to read or allocate the whole input.

Appendix B. The separate-cap direct product

We prove the separate-cap direct-product lemma used in Section 5. The input samples are independent, each has the same hard law, each individual cap holds on every branch, and the coins and side information are jointly independent of the entire tuple.

Source note. The separately capped setting goes back to Shaltiel's fair decision trees (2003). The proof below is the conditional-value argument of Drucker's Lemmas 3.1–3.2, whose headline theorem uses one global budget instead (2012, sec. 1.3 and the end of §3).

Fix the coins and side information. By joint independence the samples are still independent with law μ, and the algorithm is a deterministic tree. Take a history τ of queries and answers that occurs with positive probability. Each recorded answer constrains only its own sample, and because the tree is deterministic, every tuple that agrees with these answers asks exactly these queries. So the tuples consistent with τ form a product $E_1(\tau) \times \cdots \times E_r(\tau)$, and given τ the samples are independent with laws $\mu(\cdot \mid E_h(\tau))$.

Now measure the best success still available with the budget left. For AND on two independent fair bits with one query allowed, before querying it is $3/4$; after seeing a zero it is 1, and after seeing a one with no queries left it is $1/2$. Let $q_h(\tau)$ count the queries to U_h in τ, and let $V_h(\tau)$ be the best probability of computing $P(U_h)$ under $\mu(\cdot \mid E_h(\tau))$ with at most $L - q_h(\tau)$ further queries. If the next query reads a bit of U_h, then querying it and continuing optimally is one strategy with that allowance, so averaging over the answer b,

$$\sum_b \Pr(b \mid \tau)\, V_h(\tau b) \le V_h(\tau).$$

One answer may raise V_h; only the average cannot. For two independent uniform AND inputs with caps of one query, observing first bit 0 in U_1 gives $E_1 = \{00, 01\}$ and $E_2 = \{00, 01, 10, 11\}$; the second sample is still unrestricted. We have $V_1 = V_2 = 3/4$ at the start; reading the first bit of U_1 makes $V_1 = 1$ or $1/2$ as the bit is 0 or 1, and V_2 stays $3/4$, so the product averages to $\frac{1}{2} \cdot \frac{3}{4} + \frac{1}{2} \cdot \frac{3}{8} = \frac{9}{16}$, its starting value.

In general the other samples keep their laws and allowances, so their values do not change, and $\prod_h V_h$ has nonincreasing expectation from step to step. It starts at most β^r. At a leaf with output s, the probability of being right given τ is $\prod_h \Pr(P(U_h) = s_h \mid \tau) \le \prod_h V_h(\tau)$. Averaging over leaves and then over the coins and side information finishes the proof; the query cap and the finite sets of input positions, answers and outputs allow only finitely many deterministic trees. Arbitrary independent randomness therefore reduces to a finite mixture of these behaviours.

Return to the final gap bound in Section 5.

Appendix C. Testing the classical hypotheses

The following examples isolate the roles of independent planting, terminal lightness and the query caps. They change the partial function where stated; they are not small hard instances of the asymptotic family.

Independent planting

The identity Pr(erased zero hit) $= \mathbb{E}Z/m$ needs I to be independent of the run. Query one fixed position in each of the four columns, so that $\mathbb{E}Z = 1$ and an independent I is hit with probability $1/4$. Suppose instead that I is chosen uniformly among the columns where a zero was found, and uniformly among all four when none was found, which happens with probability $(3/4)^4 = 81/256$. By symmetry I is still uniform on its own, but now it is hit whenever some zero was found, with probability $1-(3/4)^4 = 175/256$. A uniform marginal is a weaker condition than independence from the run, and the proof needs independence.

Lightness is a joint event

Why the event must be joint. A run that stays light behaves like a capped algorithm, but it need not succeed with probability at most β^r given that it stays light. Take $P = \mathrm{AND}$ on the three promised inputs $01, 10, 11$, uniformly; any one-query algorithm is right with probability at most $\beta = 2/3$. Read the first bit, output 0 if it is 0, and otherwise read the second bit. This algorithm stays within one query exactly on input 01, where it is right, so its success given that it stays light is 1. The true statement concerns the joint event: being right **and** staying within the caps has probability $1/3 \le \beta$. This toy illustrates conditioning: it meets the one-query hard-law premise but has $R(P) = L = 1$, so it does not instantiate the strict $L < R(P)$ regime of our construction (record Appendix B.3).

Two clocks in a capped simulation

A small trace makes the cap concrete. Use the record's XOR example, which changes the partial function to $P(a, b) = a \oplus b$. Use column i as the target. In this separate toy, let the two inputs (a, b) and (c, d) be independent fair bit pairs, and set $L = 1$. The dummy algorithm first reads an outside string bit, then a. Its third query is outside the target if $a = 1$, and reads c if $a = 0$. Its fourth query proposes a bit in $B_{i,(a,0)}$.

Global time	Proposed query	Forwarded count and action
1	Outside string bit	$q = 0$; answer locally.
2	a	$q : 0 \to 1$; forward.
3, $a = 1$	Outside target	$q = 1$; answer locally.
3, $a = 0$	c	$q = 1 = L$; stop before reading.
4, $a = 1$	Cell $B_{i,10}$	$q = 1$; output label 10 without reading.

If $a = 1$, the simulator has spent its one target query learning a. It answers the third query from its outside information, then reaches the fourth query and outputs 10, read from the proposed cell position. It does not read the bit in that cell.

If $a = 0$, the proposed third query would read c, using a second target query. The simulator stops before that query and outputs the default address 00. Its cap is enforced on this branch as well.

The first branch gives the light true-cell event exactly when $a = 1, b = 0, c = d$, with probability $1/8$. The simulator is also correct on $a = 0, b = 0, c = d$, another event of probability $1/8$. Its total success probability is $1/4$, the separate-cap bound $(1/2)^2$ for two fair XOR inputs. Aborting is part of the algorithm; the probabilities are under the original input distribution, without conditioning on the query count.

Separate caps and independent information

Separate caps. Take AND on two independent inputs, each uniform on all four two-bit strings; one query to an input gives $\beta = 3/4$. A pooled budget of two queries computes the first answer exactly and guesses 0 for the second, which is right with probability $3/4 > 9/16$.

Joint independence. For the identity on fair bits X, Y with $L = 0$, the side information $X \oplus Y$ is independent of each bit separately, yet the guess $(0, X \oplus Y)$ is right whenever $X = 0$, with probability $1/2 > 1/4$.

Appendix D. Exercises and answers

These questions revisit the board, the weighted sampler and the capped predictor. The examples in Appendix C supply the independent-planting and two-clock setups. The answers follow the questions.

Projection check. For 1111 1000, calculate the signed amplitude Φ and then the projection probability. These are different quantities. The answer is separated from the attempt.

Question 1 (promise certification). On the domain-bit board of Section 4, make one row for each of column 2's four addresses. In each row name the domain and value outputs that fail and count them; no gate indices are required. (Answer.)

Question 2 (conditional sampling and overshoot). Reconstruct the cycled-board calculation in Section 3.2: after pivots $(2, 4)$, why does one amplification iteration help and a second undo the gain? An iteration uses the entire preparation and its inverse, not one bit query. Now continue the unequal-weight example with pivot 4, taking the four win probabilities against it to be $(1, 3/4, 1/2, 0)$. Find the joint success masses and conditional label law. Multiply the current weights by the comparison probabilities; do not restart from all ones. (Answer.)

Question 3 (independent planting). In the independent-planting example of Appendix C, now query two fixed distinct positions per column. How often is an independent I hit, and how often the dependent one? If later queries depend on earlier replies, which identity still holds for an independent I, and why? (Answer.)

Question 4 (two clocks and the capped event). In the two-clock trace of Appendix C, change the $a = 0$ branch so that it proposes a query in $B_{i,00}$ at time 3, before trying to read c. If the run stops after this cell query, what probability does the changed branch contribute, and what is the total probability of a light true-cell visit at either indicated time, including the unchanged $a = 1$ branch at time 4? What changes if the $a = 0$ branch reads c afterwards? Which fixed-time events must be bounded separately by the simulator lemma? (Answer.)

Question 5 (uniform constants and endpoint powers). What does $k = 4$ prove about α_{tot}? Which k makes the argument of Section 6 exclude the endpoint bound $R(f) \le C(1 + Q(f))^{3.5}$ itself? (Answer.)

Combined explanation. Using the cycled board and the planted-pair/XOR examples, explain why the quantum list can contain a winner even when a later stage fails, and why any classical light visit gives a capped address prediction. Why does an arbitrary rival's off-promise decoder law not spoil comparison with a genuine winner? For the classical route, take an integer budget $G \le c\,mL$: reconstruct the three charges, show that their sum is at most $4c + c + c/m^8$, and identify where $L \le m$, $r = 20 \log_2 m$ and averaging over the independent I enter. Explain which half of $\alpha_{\text{tot}} = 4$ these mechanisms establish and which half uses the imported universal theorem. Try this before reading the rubric. A one-route reader may quote the other route's bound.

Answers

Projection check. The first sign block is $(-1, -1, -1, -1)$ and its dot product with $(2, -2, -2, -2)$ is 4. Thus the signed amplitude is $\Phi = 4/8 = 1/2$ and projection probability is $|\Phi|^2 = 1/4$.

Question 1. String, pointer and gate tests pass. Domain bits $(0, 1)$ and value bits $(0, 0)$ make the first domain test fail always, and a value test fail wherever the address differs from 00: one failure at 00, two at 01 and 10, three at 11.

Question 2. Multiplying $(1, 0, 0, 1)$ by $(a_{14}, a_{24}, a_{34}, a_{44}) = (1, 0, 1, 0)$ gives $(1, 0, 0, 0)$. A preparation succeeds with probability $1/4 \ge \gamma$ and then yields column 1; with $\theta = \pi/6$, one Grover iteration makes success certain, as $\sin^2(3\pi/6) = 1$. A second iteration overshoots: $\sin^2(5\pi/6) = 1/4$. For the fractional example the new weights are $(1, 0, 1/4, 0)$, joint masses $(1/4, 0, 1/16, 0)$, total success $5/16$, and conditional law $(4/5, 0, 1/5, 0)$. Returning now to the cycled physical board, column 1 passes verification from the list $\{2, 4, 1\}$. The unequal-weight example specifies only sampler probabilities; certification uses the board's strings, pointers and transcripts. Columns 2 and 4 have failed string tests; the bounded-error verifier discovers those failures with its stated high probability, not with certainty.

Question 3. Even for adaptive queries, independence gives $\Pr(\text{erased zero hit}) = \mathbb{E}Z/4$ by conditioning on the complete dummy run. For the fixed two-position experiment, an independent I is hit with probability $\mathbb{E}Z/4 = 1/2$; the dependent choice misses only when no zero is found, so it is hit with probability $1 - (1/2)^4 = 15/16$. This product uses two distinct fixed positions in each column and independent zero locations. An adaptive policy preserves the complete-run identity but need not preserve that fixed-pattern product.

Question 4. First assume that the changed branch stops after the cell query. The cell query now occurs after only one target query, so the simulator reaches it and outputs 00. It is a true-cell query when $a = 0, b = 0, c = d$, with probability $(1/2)(1/2)(1/2) = 1/8$. Including the unchanged $a = 1$ branch gives total probability $1/4$. If the algorithm instead reads c afterwards, the entire $a = 0$ branch is heavy and contributes zero to the joint light-visit event. No query is deleted from the count. The remaining probability is $1/8$, from the unchanged $a = 1$ branch. In the stopping version, the fixed-time probabilities are $1/8$ at time 3 and $1/8$ at time 4; their union is $1/4$.

Question 5. $k = 4$ excludes every power strictly below 3.5, so it proves $\alpha_{\mathrm{tot}} \geq 3.5$. To exclude the endpoint inequality itself by this argument, take $k > 2/(4 - 3.5) = 4$, for example $k = 5$. At $k = 4$ and $\alpha = 3.5$, the ratio estimate has $d = 0$ and only a reciprocal logarithmic factor; it does not diverge. At $k = 5$, $d = 2 - 1/5 - 3.5/2 = 1/20 > 0$, so it does diverge.

Combined explanation. Each unlisted winner has weight at least $(1 - \varepsilon)^T$. Halving makes this much remaining weight unlikely, and a later failure preserves the list already drawn. On the losing board, predict the address from a proposed query coordinate, aborting before the first unaffordable target query. Every light true-cell visit supplies a correct prediction at its query time. For $G \leq c\,mL$ the three charges are bounded by $4cL/m$, c and $c\,mL\,m^{-10}$. Since $L \leq m$ these sum to at most $4c + c + c/m^8$. The first two charges use the independence of the planted column. In the third, sum over times and average over columns. These arguments exclude every smaller universal exponent; the deterministic quartic bound establishes that four suffices.

Appendix E. Following the formal proofs

The theorem index connects the mathematical statements to their Lean definitions and proofs. Start with the winning condition: the domain-bit board fails precisely because one of its address inputs is not promised. The family theorem uses one circuit for both complexities, and the explicit bounds give the six and 1/400 printed here.

The formal quantum argument uses the exact probability law of the measured candidate, followed by verification. That law is what the Boolean algorithm needs. The stronger assertion about retaining the whole successful physical state has a separate correspondence still to be formalized; the headline result does not assume it. The optimal-exponent theorem combines the separating family with a formal proof of the universal quartic bound. The index explains these proof routes and gives their precise scope.

References

Aaronson, Scott, and Andris Ambainis. 2018. "Forrelation: A Problem That Optimally Separates Quantum from Classical Computing." *SIAM Journal on Computing* 47 (3): 982–1038.

Aaronson, Scott, Shalev Ben-David, and Robin Kothari. 2016. "Separations in Query Complexity Using Cheat Sheets." *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, 863–76.

Aaronson, Scott, Shalev Ben-David, Robin Kothari, Shravas Rao, and Avishay Tal. 2021. "Degree vs. Approximate Degree and Quantum Implications of Huang's Sensitivity Theorem." *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 1330–42.

Ambainis, Andris, Kaspars Balodis, Aleksandrs Belovs, Troy Lee, Miklos Santha, and Juris Smotrovs. 2017. "Separations in Query Complexity Based on Pointer Functions." *Journal of the ACM* 64 (5): 32:1–24.

Ambainis, Andris, Jānis Iraids, and Martins Kokainis. 2026. *Near-Optimal Separations of Certificate Complexity from Randomized and Quantum Query Complexity*. https://arxiv.org/abs/2609.11664v3.

Bansal, Nikhil, and Makrand Sinha. 2021. "k-Forrelation Optimally Separates Quantum and Classical Query Complexity." *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 1303–16.

Ben-David, Shalev. 2015. *A Super-Grover Separation Between Randomized and Quantum Query Complexities*. https://arxiv.org/abs/1506.08106.

Ben-David, Shalev, and Robin Kothari. 2026. *Randomized Query Complexity Can Beat Certificate Complexity*. https://arxiv.org/abs/2609.15063v1.

Boyer, Michel, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. "Tight Bounds on Quantum Searching." *Fortschritte der Physik* 46 (4–5): 493–505. https://doi.org/10.1002/(SICI)1521-3978(199806)46:4/5%3C493::AID-PROP493%3E3.0.CO;2-P.

Brassard, Gilles, Peter Høyer, Michele Mosca, and Alain Tapp. 2002. "Quantum Amplitude Amplification and Estimation." *Quantum Computation and Quantum Information*, Contemporary Mathematics, vol. 305: 53–74.

Buhrman, Harry, and Ronald de Wolf. 2002. "Complexity Measures and Decision Tree Complexity: A Survey." *Theoretical Computer Science* 288 (1): 21–43.

de Wolf, Ronald. 2026. *Quantum Computing: Lecture Notes*. https://arxiv.org/abs/1907.09415v6.

Drucker, Andrew. 2012. "Improved Direct Product Theorems for Randomized Query Complexity." *Computational Complexity* 21 (2): 197–244.

Dürr, Christoph, and Peter Høyer. 1996. *A Quantum Algorithm for Finding the Minimum*. https://arxiv.org/abs/quant-ph/9607014.

Göös, Mika, Toniann Pitassi, and Thomas Watson. 2018. "Deterministic Communication vs. Partition Number." *SIAM Journal on Computing* 47 (6): 2435–50.

Høyer, Peter, Michele Mosca, and Ronald de Wolf. 2003. "Quantum Search on Bounded-Error Inputs." *Automata, Languages and Programming (ICALP 2003)*, Lecture Notes in Computer Science, vol. 2719: 291–99.

Huang, Hao. 2019. "Induced Subgraphs of Hypercubes and a Proof of the Sensitivity Conjecture." *Annals of Mathematics* 190 (3): 949–55.

OpenAI. 2026. *A Nearly Quartic Separation Between Randomized and Quantum Query Complexity*. Online source.

Shaltiel, Ronen. 2003. "Towards Proving Strong Direct Product Theorems." *Computational Complexity* 12 (1–2): 1–22.

Sherstov, Alexander A., Andrey A. Storozhenko, and Pei Wu. 2023. "An Optimal Separation of Randomized and Quantum Query Complexity." *SIAM Journal on Computing* 52 (2): 525–67.

Tal, Avishay. 2020. "Towards Optimal Separations Between Quantum and Randomized Query Complexities." *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, 228–39.

Yao, Andrew Chi-Chih. 1977. "Probabilistic Computations: Toward a Unified Measure of Complexity." *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*, 222–27.